

Explore | Expand | Enrich

<Codemithra />™

JOSEPHUS PROBLEM



TEST TIME ON COMBINATION

URL: https://forms.gle/9e9Xk6RhCqE37aTZ8

QR CODE:



JOSEPHUS PROBLEM

Introduction

This problem goes back to Josephus Flavius, a 1st century Roman historian

41 rebels are trapped in a cave, surrounded by enemy troops.They decide to commit suicide: they line up in a circle and systematically kill every other one, going around and around, until only one rebel is left -- who supposedly kills himself (fat chance).

JOSEPHUS PROBLEM

Task

Given the total number of persons n and a number k which indicates that k-1 persons are skipped and kth person is killed in circle.

The task is to choose the place in the initial circle so that you are the last one remaining and so survive.

JOSEPHUS PROBLEM

Examples

n = 5 and k = 2

The safe position is 3. Firstly, the person at position 2 is killed, then person at position 4 is killed, then person at position 1 is killed. Finally, the person at position 5 is killed. So the person at position 3 survives

n = 7 and k = 3

The safe position is 4. The persons at positions 3, 6, 2, 7, 5, 1 are killed in order, and person at position 4 survives

JOSEPHUS PROBLEM

eTHNUS
Explore | Expand | Enrich

Thinking

Clearly, we can represent the rebels by a list of numbers {1,2,...,41} and we can simulate the demise of the corresponding rebel by manipulating this list.

For example, we could just mark the dead rebels by switching item i to -i (a standard hack, and a true abuse of negative numbers).

Actually, it will be more interesting to generalize slightly: let us deal with the problem for an arbitrary number n of rebels.

So, our original list is {1,2,...,n} , and we have to make a n-1 deletions in the right place to find the survivor.

JOSEPHUS PROBLEM

Structure

The problem has following recursive structure

$$josephus(n, k) = (josephus(n - 1, k) + k-1) \% n + 1$$
$$josephus(1, k) = 1$$

After the first person (kth from beginning) is killed, n-1 persons are left.

So we call josephus(n – 1, k) to get the position with n-1 persons.

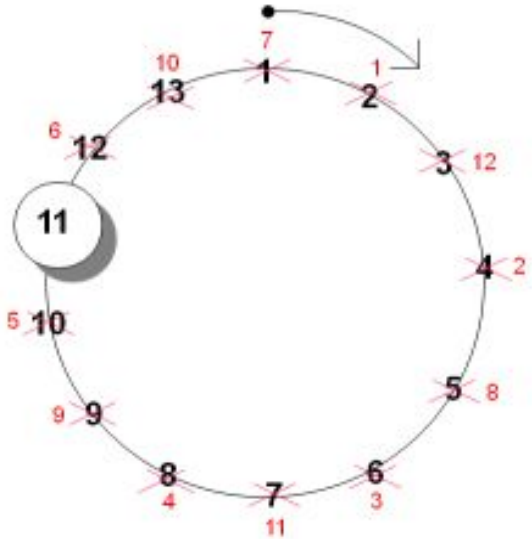
But the position returned by josephus(n – 1, k) will consider the position starting from k%n + 1.

So, we must make adjustments to the position returned by josephus(n – 1, k).

JOSEPHUS PROBLEM

Example

In the above image the execution starts from 2 and finally the free person will be 11.



JOSEPHUS PROBLEM

Program

josephus1.java
IO Format
Input
n = 14; k = 2

Output
13

Explanation
The chosen place is 13

Time Complexity
O(n)

JOSEPHUS PROBLEM

```
import java.io.*;
class EthCOde {
     static int josephus(int n, int k)
     {
     if (n == 1)
     return 1;
     else
/* The position returned by josephus(n - 1, k) is adjusted because the recursive
call josephus(n - 1, k) considers the original position k%n + 1 as position 1 */
          return (josephus(n - 1, k) + k - 1) % n + 1;
     }
     public static void main(String[] args)
     {
     int n = 14;
     int k = 2;
     System.out.println(josephus(n, k));
     }
}
```

JOSEPHUS PROBLEM

Another Solution

Let’s take an example of there are 5 people in a circle and execution starts clockwise at position 3

So there are five people – 1 2 3 4 5 and execution starts clockwise at position 3. Therefore if you kill 3 then are left with the list something like – 1 2 4 5.

Now again you have to kill the person at position 3 clockwise counting from the last killed person.

JOSEPHUS PROBLEM

Another Solution

Now you need to execute the person at position 1. So executing the person at 1 you are left with the list 2 4 5.
After executing another person you get list 2 4.

Now you are left with 2 persons in the list. So you have to calculate the position wisely to execute the person.

Therefore the calculated position would be
(starting position of the execution)%(no of people remaining in the list less than starting position)
i.e. 3%2 = 1.
So finally the winner is 4.

JOSEPHUS PROBLEM

Program

josephus2.java

IO Format
Input
n = 14; k = 2

Output
13

Explanation
The chosen place is 13

Time Complexity
O(n)

JOSEPHUS PROBLEM

```
import java.util.List;
import java.util.stream.Collectors;
import java.util.stream.IntStream;
public class JosephProblem {
     public static void main(String[]
args) {
     int winner = joseph(5, 3);
System.out.println(winner);
     }
     public static int joseph(int
noOfPeople, int remPosition) {
     int tempPos = remPosition - 1;
    int[] people = new int[noOfPeople];
   for (int i = 0; i < noOfPeople; i++) {
     people[i] = i + 1;
          }
     int iteration = noOfPeople - 1;
```

```
List<Integer> list =
IntStream.of(people).boxed().collect(Colle
ctors.toList());
     while (iteration > 0) {
list.remove(tempPos);
     tempPos += remPosition - 1;
     if (tempPos > list.size() - 1) {
    tempPos = tempPos % list.size();
}
     iteration--;
          }
     return list.get(0);
     }

}
```

<Codemithra />™

eTHNUS™
Explore | Expand | Enrich

THANK YOU

+91 78150 95095 | codemithra@ethnus.com | www.codemithra.com